

DIPLOMADO · MÓDULO 3 · BACKEND Y APIS REST

EVALUACIÓN 1 · COPA LIBERTADORES

🎯 OBJETIVO

Qué vas a construir y qué se está evaluando.

OBJETIVO GENERAL

Crear desde cero un proyecto de Node con Express y construir una **API REST** que responde consultas sobre los **clubes** de la Copa Libertadores, organizados por **país** y con los años de los títulos que ganaron; y **explicar tus decisiones** en un video corto.

OBJETIVOS DE APRENDIZAJE

1. **Iniciar un proyecto** de Node: `package.json`, dependencias, módulos ES y un script que reinicia el servidor solo.
2. Construir rutas con Express usando **parámetros de ruta** (`:id`) y **de consulta** (`?pais=`).
3. Resolver búsquedas con los **métodos de array** (`find` , `filter`), incluida una **búsqueda anidada** entre dos arrays relacionados.
4. Responder con el **código de estado correcto** y errores claros.

> ⏱️ **ESTÁ PENSADA PARA 60 MINUTOS**
>
> Más un video de 3 a 5 minutos. Puedes hacerla cuando quieras **hasta el martes 20 de octubre**, de una vez o por partes en distintos días: como te acomode.

🛠️ CHULETA · CREAR EL PROYECTO DESDE CERO

Paso a paso. Necesitas Node 20 o más nuevo (`node -v`) y yarn (`yarn -v`).

1. Crea la carpeta y ábrela en VS Code

```
mkdir evaluacion-1-libertadores
cd evaluacion-1-libertadores
code .
```

Desde aquí, todos los comandos van en la **terminal de VS Code** (`Ctrl + ñ` o *Terminal → New Terminal*), dentro de esta carpeta.

2. Crea el `package.json`

```
yarn init -y
```

El `-y` acepta todas las preguntas con sus valores por defecto. El `package.json` es la ficha del proyecto: su nombre, sus dependencias y sus scripts.

3. Instala las dependencias

```
yarn add express cors
```

Se crea la carpeta `node_modules` (el código de las librerías) y las dos quedan anotadas en `"dependencies"` del `package.json`.

4. Ajusta el `package.json` : módulos y el script `dev`

Ábrelo y agrégale dos cosas: `"type": "module"` y el bloque `"scripts"`. Debe quedar parecido a esto (las versiones pueden variar):

```
{
  "name": "evaluacion-1-libertadores",
  "version": "1.0.0",
  "main": "index.js",
  "license": "MIT",
  "type": "module",
  "scripts": {
```

```
    "dev": "node --watch server.js"
  },
  "dependencies": {
    "cors": "^2.8.6",
    "express": "^5.3.0"
  }
}
```

- `"type": "module"` → te deja usar `import` y `export`.
- `node --watch` → **reinicia el servidor solo cada vez que guardas**. No tienes que bajarlo y volver a levantarlo con cada cambio.

5. Crea el `.gitignore`

Un archivo llamado exactamente `.gitignore`, con:

```
node_modules
.env
```

`node_modules` **nunca** se sube a GitHub: pesa cientos de megas y cualquiera lo regenera con `yarn`.

6. Crea `datos.js` y pega los datos

Copia los arreglos de la sección 📦 **Los datos** (más abajo, con el botón *Copiar*) y pégalos tal cual en un archivo nuevo `datos.js`.

7. Crea `server.js` y levántalo

```
import express from 'express'
import cors from 'cors'
import { paises, clubes } from './datos.js'

const app = express()
app.use(cors())

const PORT = 3000

// 👉 aquí van tus rutas

app.listen(PORT, () => {
```

```
  console.log(`🏆 API escuchando en http://localhost:${PORT}`)
})
```

```
yarn dev
```

Si ves el mensaje 🏆, está funcionando. **Deja esa terminal abierta**: cada vez que guardes `server.js`, se reinicia sola.

8. Al terminar: súbelo a GitHub

En github.com/new crea un repositorio **público y vacío** (sin README ni .gitignore). Después, en tu terminal (abre otra, la del `yarn dev` está ocupada):

```
git init
git add .
git commit -m "Evaluación 1 · API de la Copa Libertadores"
git branch -M main
git remote add origin https://github.com/TU-USUARIO/TU-REPO.git
git push -u origin main
```

Antes del `git add .`, revisa con `git status` que `node_modules` **no** aparezca en la lista.

ASÍ DEBE QUEDAR TU PROYECTO

```
evaluacion-1-libertadores/
├── node_modules/     ← existe en tu PC, pero NO se sube
├── .gitignore
├── datos.js
├── package.json
├── README.md         ← cómo levantarlo + el enlace a tu video
├── server.js
└── yarn.lock
```

📦 LOS DATOS

16 clubes de 5 países, con los años de las Copas Libertadores que ganaron (títulos reales hasta 2025). Cópialos tal cual en `datos.js`.

```
// Un país agrupa clubes
export const paises = [
  { id: 1, nombre: 'Argentina' },
  { id: 2, nombre: 'Brasil' },
  { id: 3, nombre: 'Chile' },
  { id: 4, nombre: 'Uruguay' },
  { id: 5, nombre: 'Paraguay' },
]

// Cada club pertenece a UN país (por su id).
// `titulos` = los años de las Copas Libertadores que ganó.
export const clubes = [
  { id: 1,  nombre: 'Boca Juniors',          paisId: 1, titulos: [1977, 1978, 2000, 2001, 2003, 2007]
},
  { id: 2,  nombre: 'River Plate',           paisId: 1, titulos: [1986, 1996, 2015, 2018] },
  { id: 3,  nombre: 'Independiente',         paisId: 1, titulos: [1964, 1965, 1972, 1973, 1974, 1975,
1984] },
  { id: 4,  nombre: 'Racing Club',           paisId: 1, titulos: [1967] },
  { id: 5,  nombre: 'São Paulo',             paisId: 2, titulos: [1992, 1993, 2005] },
  { id: 6,  nombre: 'Santos',                paisId: 2, titulos: [1962, 1963, 2011] },
  { id: 7,  nombre: 'Grêmio',                paisId: 2, titulos: [1983, 1995, 2017] },
  { id: 8,  nombre: 'Botafogo',              paisId: 2, titulos: [2024] },
  { id: 9,  nombre: 'Colo-Colo',             paisId: 3, titulos: [1991] },
  { id: 10, nombre: 'Universidad de Chile',  paisId: 3, titulos: [] },
  { id: 11, nombre: 'Universidad Católica',  paisId: 3, titulos: [] },
  { id: 12, nombre: 'Cobreloa',              paisId: 3, titulos: [] },
  { id: 13, nombre: 'Peñarol',               paisId: 4, titulos: [1960, 1961, 1966, 1982, 1987] },
  { id: 14, nombre: 'Nacional',              paisId: 4, titulos: [1971, 1980, 1988] },
  { id: 15, nombre: 'Olimpia',               paisId: 5, titulos: [1979, 1990, 2002] },
  { id: 16, nombre: 'Cerro Porteño',         paisId: 5, titulos: [] },
]
```

> 📌 **FÍJATE EN CÓMO SE RELACIONAN**
>
> El club **no guarda** el nombre de su país: guarda su `paisId`. Para responder "los clubes de Chile" tienes que **resolver esa referencia**: es la búsqueda anidada del repaso.

🧰 REPASO RÁPIDO DE LAS HERRAMIENTAS

Con otros datos, a propósito: te muestran cómo funciona cada método, no cómo resolver tu prueba.

```
const libros = [
  { titulo: 'Rayuela',   autorId: 1 },
  { titulo: 'Ficciones', autorId: 2 },
  { titulo: 'El Aleph',  autorId: 2 },
]
const autores = [
  { id: 1, nombre: 'Cortázar' },
  { id: 2, nombre: 'Borges' },
]
```

🔎 find — encontrar UNO

```
libros.find(l => l.titulo === 'El Aleph')   // { titulo: 'El Aleph', ... }
libros.find(l => l.titulo === 'Nada')       // undefined  ← valida antes de usarlo
```

🧹 filter — quedarte con VARIOS

```
libros.filter(l => l.autorId === 2)         // [ Ficciones, El Aleph ]
libros.filter(l => l.autorId === 99)        // []  ← vacío, NO undefined
```

🕵️ Búsqueda anidada — cuando el dato está en OTRO array

El libro no guarda el nombre del autor: guarda su `autorId`. Para "los libros de Borges" hay que dar **dos saltos**.

```
const autor = autores.find(a => a.nombre === 'Borges')   // 1) resolver la referencia
if (!autor) { /* 2) ¿y si no existe? → 404 */ }
const suyos = libros.filter(l => l.autorId === autor.id) // 3) ahora sí, filtrar
```

🧭 Parámetros: de ruta y de consulta

```
// GET /api/libros/2         → req.params.id === '2'   (llega como TEXTO)
// GET /api/libros?autor=X   → req.query.autor === 'X'
```

```
Number(req.params.id)       // 2  ← conviértelo antes de comparar con ===

// sin importar mayúsculas:
'Borges'.toLowerCase() === 'BORGES'.toLowerCase()   // true
```

✅ **UN ARRAY VACÍO NO ES UN ERROR**

`filter` devuelve `[]` cuando nada cumple; `find` devuelve `undefined` cuando no encuentra. Tu API debe tratarlos distinto: una lista vacía es un `200`; algo que no existe, un `404`.

📋 LAS RUTAS

Lo que tu API debe responder. Léelo completo antes de escribir código.

RUTA	QUÉ DEBE HACER
`GET /api/clubes`	Los 16 clubes.
`GET /api/clubes/:id`	Un club por su id. Si no existe → `404`.
`GET /api/clubes?pais=Chile`	🔍 **Búsqueda anidada.** Va **en la misma ruta** que `/api/clubes`: si viene `?pais=`, busca primero el país por su nombre y después filtra los clubes por su id. Si el país no existe → `404`.
`GET /api/titulos/:club`	🔍 **Buscar por nombre.** Devuelve el array de años de ese club (`/api/titulos/colo-colo` → `[1991]`). Si el club no existe → `404`.

> ⚠️ **"NO EXISTE" Y "NO TIENE" SON COSAS DISTINTAS**
>
> `/api/titulos/universidad de chile` → el club **existe**, solo que no ha ganado: `200` **con** `[]`.
> `/api/titulos/narnia` → ese club **no existe**: `404`.

> 📌 **MAYÚSCULAS**
>
> `?pais=Chile`, `?pais=chile` y `?pais=CHILE` deben responder lo mismo. Igual con el nombre del club en `/api/titulos/:club`. (Las tildes sí cuentan: `Peñarol` se escribe con ñ.)

Códigos de estado y errores

- `200` cuando encuentras lo que se pide (aunque sea una lista vacía), `404` cuando no existe.
- Los errores siempre como JSON: `{ "error": "No existe el club 99" }`. Nunca un `200` con un error adentro.
- El servidor **no se cae** con un id o un nombre raro.

🌟 Desafíos extra (opcionales)

- `GET /api/titulos`: todos los años de todos los clubes en una sola lista, ordenada (`flatMap` + `sort`).
- `GET /api/estadisticas`: total de clubes, total de títulos y títulos por país (`reduce`).

No tienen puntaje propio. Si algún criterio te quedó corto, un extra bien resuelto puede subirlo un nivel.

🚀 ENTREGA

Hasta el martes 20 de octubre. En el LMS entregas el enlace a tu repositorio.

1. Tu repositorio en GitHub, público

- El proyecto completo, **sin** `node_modules`.
- Un `README.md` breve: qué comandos lo levantan (`yarn` y `yarn dev`), en qué puerto queda, y **el enlace a tu video**.

2. Un video de 3 a 5 minutos

Súbelo a YouTube (puede ser **no listado**) y **pega el enlace en tu** `README.md`. Pruébalo en una ventana de incógnito: debe abrirse sin pedir permisos.

QUÉ TIENE QUE MOSTRAR

1. **Tu proyecto:** el `package.json` (el `"type"` y el script `dev`) y el servidor corriendo con `yarn dev`. Guarda un cambio y muestra que **se reinicia solo**.
2. **Tu API en Postman**, con los códigos de estado a la vista:
   - `/api/clubes/9` y `/api/clubes/99`
   - `?pais=Chile` y `?pais=narnia`
   - `/api/titulos/colo-colo`, `/api/titulos/universidad de chile` y `/api/titulos/narnia`
3. **Una decisión explicada por ti**, con el código en pantalla: cómo resolviste `?pais=` (la búsqueda anidada) y por qué `[]` y `404` no son lo mismo. No leas el código línea por línea: explica *por qué* lo hiciste así.

> 📌 **REQUISITOS DEL VIDEO**
>
> Tu voz explicando, y el código y Postman deben **leerse** en pantalla. Una toma continua está bien: no hace falta editarlo.
> **Si te pasas de los 5 minutos, no te preocupes:** no pierdas tiempo re-editando para recortarlo. El video se ve completo, dure lo que dure.

📐 RÚBRICA

Cada criterio se evalúa en uno de cuatro niveles. Total: 100 puntos.

CRITERIO	PTS	4 · DESTACADO	3 · LOGRADO	2 · EN DESARROLLO	1 · INSUFICIENTE
Iniciar el proyecto package.json · dependencias · --watch · .gitignore	15	Tiene `"type": "module"`, `express` y `cors` en las dependencias, el script `dev` con `--watch` y el `.gitignore`. Levanta con `yarn` + `yarn dev`. **15 pts**	Levanta con `yarn dev`, pero falta un detalle (ej. sin `--watch` o sin `.gitignore`). **11 pts**	Hay que corregir algo para que levante, o faltan varias piezas. **7 pts**	No levanta, o subió `node_modules`. **0 pts**
Las rutas /clubes · /clubes/:id · ?pais · /titulos/:club	35	Las cuatro funcionan, incluidos los casos borde: mayúsculas, `[]` vs `404`, país inexistente. **35 pts**	Las cuatro funcionan; falla algún caso borde. **26 pts**	Funcionan dos o tres. **17 pts**	Funciona una o ninguna. **0 pts**
Métodos de array y búsqueda anidada find · filter	20	Usa el método adecuado a cada problema. La búsqueda anidada es limpia: resuelve el país y después filtra. **20 pts**	Usa los métodos correctamente, aunque alguna solución sea más larga de lo necesario. **15 pts**	Usa métodos de array pero elige mal alguno, o la búsqueda anidada falla en algún caso. **10 pts**	Usa ciclos `for` en vez de los métodos, o no hay búsqueda anidada. **0 pts**
Códigos de estado y errores 200 · 404 · { error }	10	Cada respuesta usa el status exacto, los errores traen un mensaje claro en JSON y el servidor nunca se cae. **10 pts**	Status correctos en los casos principales; los errores traen mensaje. **8 pts**	Confunde algunos status (ej. `200` con un error adentro, o `404` para una lista vacía). **5 pts**	Responde siempre `200`, o el servidor se cae. **0 pts**
Entrega Repo público · README	5	Repo público, con un README que dice cómo levantarlo y tiene el enlace al	Repo público; al README le falta cómo levantarlo o	Repo público sin README. **2 pts**	No hay repo, o es privado. **0 pts**

		video. **5 pts**	el enlace. **4 pts**		
Video · demostración Proyecto andando + Postman	7	Muestra el `package.json`, el reinicio con `--watch` y todos los casos en Postman con los status a la vista. **7 pts**	Muestra la mayoría de lo pedido. **5 pts**	Muestra algunas rutas, o no se alcanza a leer la pantalla. **3 pts**	No muestra la API funcionando. **0 pts**
Video · explicación El *por qué*, no el *qué*	8	Explica con sus palabras la búsqueda anidada y la diferencia entre `[]` y `404`. Se nota que el código es suyo. **8 pts**	Explica las dos cosas y se entiende, aunque breve. **6 pts**	Describe el código sin justificar ("aquí hago un filter..."), o explica solo una. **4 pts**	No explica, o la explicación no corresponde con su código. **0 pts**
Total	100	El proyecto y la API valen 85 puntos; el video, 15. El video demuestra el código: no lo reemplaza.			

Instituto Profesional San Sebastián · Diplomado · Módulo 3 — Backend y APIs REST